



Deploy Foody to your workspace

Work hard, skip hangry. · self-host guide

Socket Mode · no public URL · ~5 min

FOR ENGINEERS · CLONE → MANIFEST → 3 TOKENS → RUN

Run your own Foody from the repo github.com/tdries/Foody-Slack-Hackathon-Submission

1 Clone & install

Node 22+. No build step — it runs TypeScript directly via `tsx`.

```
# Node 22+
git clone https://github.com/tdries/\
  Foody-Slack-Hackathon-Submission.git
cd Foody-Slack-Hackathon-Submission
npm install
cp .env.example .env
```

2 Create the Slack app

Build it from the bundled manifest — every scope, event & Socket-Mode setting is pre-declared.

```
# api.slack.com/apps → Create New App
From a manifest → pick your workspace
paste docs/slack-manifest.yml
→ Create → Install to Workspace
```

3 Grab three tokens

Copy each into `.env`. Confirm **Socket Mode = On** (the manifest sets it).

```
# OAuth & Permissions
SLACK_BOT_TOKEN=xoxb-...
# Basic Info → App-Level Tokens (connections:write)
SLACK_APP_TOKEN=xapp-...
# Basic Info → Signing Secret
SLACK_SIGNING_SECRET=...
```

4 Configure runtime

For a server, set `FOODY_DISABLE_LIVE=1` — deterministic mock data, no scraping, no Chrome.

```
FOODY_DISABLE_LIVE=1 # server-safe mock
FOODY_TAKEAWAY_LOCALE=be-en # or nl, ...
FOODY_CHANNELS=      # C0...,C0... empty=all
FOODY_LOG_LEVEL=info
```

5 Run & invite

Dials out to Slack over a WebSocket — nothing inbound. Then in a channel:

```
npm run start:demo # mock, server-safe
# or: npm run dev:slack (live + watch)

in Slack: /invite @Foody
          let's eat something
```

6 Host it always-on

Ships with a `Dockerfile` + `render.yaml`. Deploy as a **background worker** (no port).

```
# Render: New → Blueprint → this repo
# set the 3 SLACK_* secrets in dashboard
docker build -t foody . && \
docker run --env-file .env foody
```

Bot scopes & events (from the manifest)

`chat:write` · post & update Block Kit · `reactions:read/write` · pre-react + read the shared cart

`channels/groups/im/mpim:history` · hear the trigger phrase ·

`users:read` · address-book greeting

message.channels message.groups message.im message.mpim
reaction_added reaction_removed

Notes

Real takeaway.com checkout is opt-in. Leave

`FOODY_DISABLE_LIVE=1` for any hosted/shared instance — the live cart drives *your* signed-in Chrome over the DevTools port and is layout-sensitive. The Slack flow, shared cart & receipt work fully on mock data.

Other markets: any takeaway.com locale sharing the same DOM works via `FOODY_TAKEAWAY_LOCALE`. Just Eat / Lieferando / Uber Eats need their own adapter in `src/scrape-live.ts`.



No LLM, no webhooks, no public URL.

Deterministic intent matching over Socket Mode · debug by hand: `npm run foody -- status <user>`

[README.md](#) · [DEMO.md](#) →